

TUGAS PENDAHULUAN

MODUL 13

2311104004

Marsella dwi julianti

SE07-01

Link Github:

Source code dan penjelasan:

https://github.com/marsellacell/KPL_MARSELLADWIJULIANTI_2311104004_SE0701/tree/marsella/13_DESIGN_PATTERN/TP

```
1  class Subject {
2      constructor() {
3          this.observers = [];
4          this.state = '';
5      }
6
7      subscribe(observer) {
8          this.observers.push(observer);
9      }
10
11     unsubscribe(observer) {
12         this.observers = this.observers.filter(obs => obs !== observer);
13     }
14
15     notify() {
16         this.observers.forEach(observer => observer.update(this.state));
17     }
18
19     setState(newState) {
20         console.log(`Subject: Mengubah state ke "${newState}"`);
21         this.state = newState;
22         this.notify();
23     }
24 }
25
26 // Observer
27 class Observer {
28     constructor(name) {
29         this.name = name;
30     }
31
32     update(state) {
33         console.log(`${this.name} menerima notifikasi: ${state}`);
34     }
35 }
36
37 // MAIN
38 const subject = new Subject();
39
40 const observer1 = new Observer("Observer 1");
41 const observer2 = new Observer("Observer 2");
42
43 subject.subscribe(observer1);
44 subject.subscribe(observer2);
45
46
```

Penjelasan Observer Pattern

A. Contoh Kondisi Penggunaan Observer Pattern

Observer Pattern dapat digunakan ketika suatu objek (disebut *subject*) perlu memberi tahu objek-objek lainnya (*observers*) saat terjadi perubahan.

Contoh:

Pada aplikasi portal berita online, saat ada berita baru dipublikasikan, semua pengguna yang mengikuti topik tertentu (misalnya "Teknologi" atau "Politik") akan langsung menerima notifikasi.

Di sini, portal berita bertindak sebagai *subject*, dan para pengguna sebagai *observers*.

B. Langkah-langkah Mengimplementasikan Observer Pattern

1. Buat *Subject* kelas
Menyimpan data dan daftar observer. *Subject* menyediakan fungsi untuk menambah, menghapus, dan memberi tahu observer.
2. Tambahkan fungsi *subscribe()* dan *unsubscribe()*
Untuk mengatur daftar observer yang ingin mengikuti update dari *subject*.
3. Implementasikan fungsi *notify()*
Saat state pada *subject* berubah, fungsi ini akan memanggil method *update()* pada setiap observer.
4. Buat *Observer* kelas
Setiap observer memiliki method *update()* yang akan menerima data terbaru dari *subject*.
5. Gunakan *subject* di kode utama (*main*)
Tambahkan observer ke *subject*, lalu ubah state pada *subject* untuk memicu pemberitahuan ke observer.

C. Kelebihan dan Kekurangan Observer Pattern

Kelebihan:

- Loose coupling: *Subject* dan observer tidak saling tergantung secara langsung, sehingga memudahkan pengembangan dan pemeliharaan.
- Fleksibel dan dinamis: Mudah menambah atau menghapus observer tanpa mempengaruhi sistem utama.
- Realtime update: Cocok untuk sistem yang membutuhkan notifikasi otomatis.

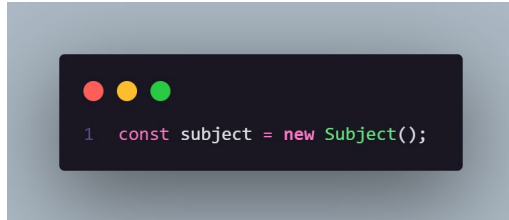
Kekurangan:

- Sulit dilacak: Jika banyak observer, bisa membingungkan untuk mengetahui siapa yang berlangganan atau menerima update.
- Overhead performa: Terlalu banyak observer dapat membuat sistem menjadi berat dan lambat.

- Risiko efek samping: Perubahan kecil di subject bisa memicu reaksi besar di observer jika tidak hati-hati.

D. Jelaskan tiap baris kode di method utama / “main”

Berikut ini versi penjelasan tiap baris kode utama (main) dalam observer.js:



```
1 const subject = new Subject();
```

Code diatas untuk membuat objek subject sebagai pusat data yang diamati (publisher).



```
1 const observer1 = new Observer("Observer 1");
2 const observer2 = new Observer("Observer 2");
```

Code diatas untuk membuat dua observer dengan nama berbeda yang siap menerima update dari subject.



```
1 subject.subscribe(observer1);
2 subject.subscribe(observer2);
```

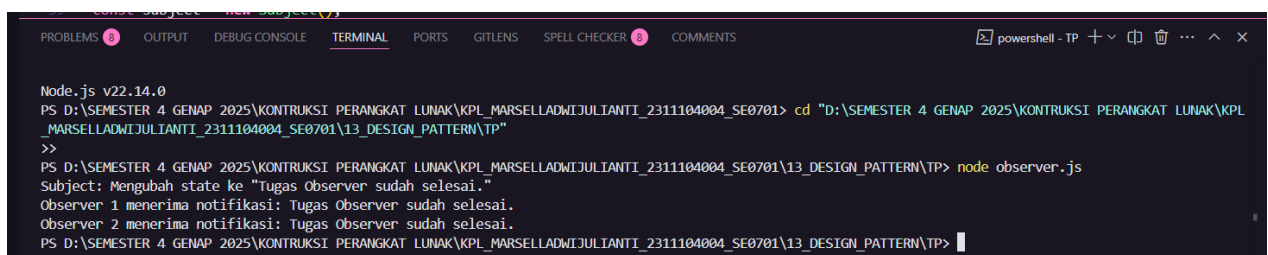
Code diatas untuk mendaftarkan observer ke dalam subject menggunakan subscribe() agar bisa menerima notifikasi.



```
1 subject.setState("Tugas Observer sudah selesai.");
```

Code diatas untuk mengubah state subject dan secara otomatis memanggil notify() untuk mengirim notifikasi ke semua observer yang telah subscribe.

Output:



```
Node.js v22.14.0
PS D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701> cd "D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701\13_DESIGN_PATTERN\TP"
>>
PS D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701\13_DESIGN_PATTERN\TP> node observer.js
Subject: Mengubah state ke "Tugas Observer sudah selesai."
Observer 1 menerima notifikasi: Tugas Observer sudah selesai.
Observer 2 menerima notifikasi: Tugas Observer sudah selesai.
PS D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701\13_DESIGN_PATTERN\TP>
```